COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 1010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	08/11/26	**DATE SUBMITTED:**	08/11/26
NAME:	Macasaet John David C.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. ___ HOA 5.1: QUEUES

1. PROCEDURE OUTPUT

CODE

```
LOA > main.cpp > ...
1   #include <iostream>
2   #include <queue>
3
4   void display(std::queue<char> copyQ);
5   int main(){
6
7       //create an object
8       std::queue<char> myQ;
9
10
11
12      //use the enqueue operation
13      myQ.push('J');
14      myQ.push('O');
15      myQ.push('H');
16      myQ.push('N');
17      std::cout<<"current front is: " <<myQ.front()<<std::endl;
18      std::cout<<"current back is: " <<myQ.back()<<std::endl;
19
20
21      display(myQ);
22      //use the dequeue operation
23      myQ.pop();
24      display(myQ);
25      myQ.pop();
26      display(myQ);
27      myQ.pop();
28      display(myQ);
29      myQ.pop();
```

OUTPUT

```
n.cpp -o main } ; if ($?) { .\main }
current front is: J
current back is: N
 J  O  H  N
 O  H  N
 H  N
 N
is the queue empty? 1
PS C:\Users\LENOVO\Downloads\MACASAET_QUEUE\ILOA>
```

```
LOA > main.cpp > ...
 5  int main(){
30      //check if the queue is empty
31      std::cout<<"is the queue empty? "<<myQ.empty()<<std::endl;
32      return 0;
33  }
34  void display(std::queue<char> copyQ){
35      //create a copy of the queue
36      std::queue<char> temp = copyQ;
37      //loop until empty
38      while(!temp.empty()){
39          //display the front
40          std::cout<<" "<<temp.front()<<" ";
41          //dequeue the front
42          temp.pop();
43      }
44      //add a new line
45      std::cout<<std::endl;
46  }
```

TABLE 1: ILOA

CODE	OUTPUT

HEADER FILE:
`LOB > C queuell.h > displayQ<T>(qNode<T>**, qNode<T>**)`
	`PS C:\Users\LENOVO\Downloads\MACASA`
	`cpp -o main } ; if ($?) { .\main }`
	`front: J back: J`
	`front: J back: O`
	`front: J back: H`
	`front: J back: N`
	
	`Testing of displayAll operator`
	`J O H N`
	
	`Testing the dequeue operator`
	`front: O back: N`
	`front: H back: N`
	`front: N back: N`
	`The queue is empty`
	
	`Testing of isEmpty operator`
	`The queue is empty`
	`PS C:\Users\LENOVO\Downloads\MACASA`

```
LOB > C queuell.h > displayQ<T>(qNode<T>**, qNode<T>**)
 1  #ifndef QUEUELL_H
 2  #define QUEUELL_H
 3
 4
 5  //create a class node
 6  template <typename T>
 7  class qNode{
 8      public:
 9          T data;
10          qNode* next;
11  };
12  //new node creator
13  template <typename T>
14  qNode<T>* new_node(T newData){
15      //allocate a space for new node
16      qNode<T>* newNode = new qNode<T>;
17      //store the new data to the new node
18      newNode -> data = newData;
19      //point the newNode to the next
20      newNode -> next = nullptr;
21
22      return newNode;
23  };
24  //enqueue
25  template <typename T>
26  void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
27      //create a new node
28      qNode<T>* newNode = new_node(newData);
29      //check if the queue is empty
```

```
#ifndef QUEUELL_H
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
    //check if the queue is empty
    if((*frontPtr) == nullptr && (*backPtr) == nullptr){
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
        return;
    }
    //if not then insert at the back
    (*backPtr) -> next = newNode;
    (*backPtr) = newNode;
};
//dequeue
template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
    //check if the queue is empty
    if((*frontPtr) == nullptr && (*backPtr) == nullptr){
        std::cout<<"The queue is empty"<<std::endl;
        return;
    }
    //create a temp variable to stor the node to be deleted
    qNode<T>* deleteTemp = nullptr;

    //assign the current front to the deleteTemp
    deleteTemp = (*frontPtr);
    //check if the queue has only 1 item
    if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
        //means only 1 item is in the queue
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
```

```
    if((*frontPtr)->next == nullptr && (*backPtr)->next == n
        delete deleteTemp;
        return;
    }

    (*frontPtr) = (*frontPtr)->next;
    deleteTemp->next = nullptr;
    //delete the node
    delete deleteTemp;
};
//display all
template <typename T>
void displayQ(qNode<T>** frontPtr,qNode<T>** backPtr ){
    //check if queue is empty
    qNode<T>* temp = (*frontPtr);
    if((*frontPtr) == nullptr && (*backPtr) == nullptr){
        std::cout<<"The queue is empty"<<std::endl;
        return;
    }
    //display all items
    while((*frontPtr) != nullptr){
        std::cout<<(*frontPtr)->data;
        if((*frontPtr)->next != nullptr){
            std::cout<<" ";
        }
        (*frontPtr) = (*frontPtr)->next;
    }
```

```
    if((*frontPtr) == nullptr){
      (*frontPtr) = temp;
    }

    std::cout<<std::endl;
};
//isEmpty
template <typename T>
bool isEmpty(qNode<T>** frontPtr,qNode<T>** backPtr){
    return ((*frontPtr) == nullptr && (*backPtr) == nullptr);
}

#endif
```

MAIN.CPP FILE:



```cpp
#include <iostream>
#include "queuell.h"

int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue('J', &front, &back);
    std::cout<<"front: "<< front->data << " "<<"back: "<<back->data<<std::endl;
    enqueue('O', &front, &back);
    std::cout<<"front: "<< front->data << " "<<"back: "<<back->data<<std::endl;
    enqueue('H', &front, &back);
    std::cout<<"front: "<< front->data << " "<<"back: "<<back->data<<std::endl;
    enqueue('N', &front, &back);
    std::cout<<"front: "<< front->data << " "<<"back: "<<back->data<<std::endl;

    //testing of displayall
    std::cout<<"\nTesting of displayAll operator\n";
    displayQ(&front, &back);
    std::cout<<std::endl;

    std::cout<<"Testing the dequeue operator\n";
    dequeue(&front, &back);
    std::cout<<"front: "<< front->data << " "<<"back: "<<back->data<<std::endl;
    dequeue(&front, &back);
    std::cout<<"front: "<< front->data << " "<<"back: "<<back->data<<std::endl;
    dequeue(&front, &back);
    std::cout<<"front: "<< front->data << " "<<"back: "<<back->data<<std::endl;
    dequeue(&front, &back);
```

```cpp
    dequeue(&front, &back);
    std::cout<<std::endl;

    std::cout<<"Testing of isEmpty operator\n";
    if(isEmpty(&front, &back)){
        std::cout<<"The queue is empty\n";
    }else{
        std::cout<<"The queue is not empty\n";
    }
    return 0;
}
```

TABLE 2: ILOB (Linked List-Based Queue)

CODE	OUTPUT

Header File:

```
ILOB > C queuearr.h > queueArr<T> > operator=(const queueArr &)
1   #ifndef QUEUEARR_H
2   #define QUEUEARR_H
3   # include <iostream>
4
5   // Array-Based Circular Queue
6   template <typename T>
7   class queueArr {
8       private:
9           // POINTER TO DYNAMICALLY ALLOCATE ARRAY
10          T* q_arr;
11
12          // MAXIMUM NUMBER OF ELEMENTS A QUEUE CAN HOLD
13          size_t q_capacity;
14
15          // CURRENT NUMBER OF ELEMENTS IN THE QUEUE
16          size_t q_size;
17
18          // INDEX OF THE FRONT ELEMENT IN THE QUEUE
19          int q_front;
20
21          // INDEX OF THE BACK ELEMENT IN THE QUEUE
22          int q_back;
23
24      public:
25
26      // CONSTRUCTOR
27      queueArr(size_t capacity) : q_capacity(capacity), q_size(0), q_front(0), q_back(-1) {
28          // DYNAMICALLY ALLOCATE MEMORY FOR THE QUEUE ARRAY
29          q_arr = new T[q_capacity];
```

```
27      queueArr(size_t capacity) : q_capacity(capacity), q_size(0), q_front(0), q_back(-1) {
30      }
31
32      // COPY CONSTRUCTOR
33      queueArr(const queueArr& other) : q_capacity(other.q_capacity), q_size(other.q_size), q_front(other.q_front), q_b
34          // DYNAMICALLY ALLOCATE MEMORY FOR THE QUEUE ARRAY
35          q_arr = new T[q_capacity];
36          // COPY ELEMENTS FROM OTHER QUEUE
37          for (size_t i = 0; i < q_size; ++i) {
38              q_arr[(q_front + i) % q_capacity] = other.q_arr[(other.q_front + i) % other.q_capacity];
39          }
40      }
41
42      // COPY ASSIGNMENT OPERATOR
43      queueArr& operator=(const queueArr& other) {
44          if (this != &other) {
45              // DEALLOCATE CURRENT ARRAY
46              delete[] q_arr;
47
48              // COPY CAPACITY AND SIZE
49              q_capacity = other.q_capacity;
50              q_size = other.q_size;
51              q_front = other.q_front;
52              q_back = other.q_back;
53
54              // DYNAMICALLY ALLOCATE MEMORY FOR THE QUEUE ARRAY
55              q_arr = new T[q_capacity];
```

```
44          if (this != &other) {
56              // COPY ELEMENTS FROM OTHER QUEUE
57              for (size_t i = 0; i < q_size; ++i) {
58                  q_arr[(q_front + i) % q_capacity] = other.q_arr[(other.q_front + i) % other.q_capacity];
59              }
60          }
61          return *this;
62      }
63
64      // DESTRUCTOR
65      ~queueArr() {
66          // DEALLOCATE MEMORY FOR THE QUEUE ARRAY
67          delete[] q_arr;
68      }
69
70      // QUEUE OPERATIONS
71      bool Empty();
72      bool Full();
73      size_t Size();
74      void Clear();
75      T Front();
76      T Back();
77      void enqueue(T value);
78      T dequeue();
79  };
```

```
PS C:\Users\LENOVO\Downloads\MACASAET_
rr.cpp -o mainarr } ; if ($?) { .\maina
Front: 10
Back: 10
Size: 1
Front: 10
Back: 20
Size: 2
Front: 10
Back: 40
Size: 4
Front: 10
Back: 50
Size: 5
Queue is Full
Tetsing Dequeue
removed: 10
removed: 20
Testing Clear
is the queue empty? yes
PS C:\Users\LENOVO\Downloads\MACASAET_
```

```cpp
// Empty()
template <typename T>
bool queueArr<T>::Empty() {
    // CHECK IF EMPTY
    return q_size == 0;
}

// Full()
template <typename T>
bool queueArr<T>::Full() {
    // CHECK IF REACHES MAX CAPACITY
    return q_size == q_capacity;
}

// Size()
template <typename T>
size_t queueArr<T>::Size() {
    // RETURN CURRENT SIZE OF QUEUE
    return q_size;
}

// Front()
template <typename T>
T queueArr<T>::Front() {
    // Check if the queue is empty
    if (Empty()) {
        std::cout << "Queue is Empty\n";
        return T(); // Return default value of type T
    }
    // Return the front
    return q_arr[q_front];
}

// Back()
template <typename T>
T queueArr<T>::Back() {
    // Check if the queue is empty
    if (Empty()) {
        std::cout << "Queue is Empty\n";
        return T(); // Return default value of type T
    }
    // Return the back
    return q_arr[q_back];
}

// Clear()
template <typename T>
void queueArr<T>::Clear() {
    // Reset the queue to its initial state
    q_size = 0;
    q_front = 0;
    q_back = -1; // Set back to -1 since there are no elemen
}
```

```cpp
template <typename T>
void queueArr<T>::enqueue(T value) {
    // Check if the queue is full
    if (Full()) {
        std::cout << "Queue is Full\n";
        return;
    }
    // Increment the back
    q_back = (q_back + 1) % q_capacity;
    // Add the new value to the back of the queue
    q_arr[q_back] = value;
    q_size++;
}
// Dequeue()
template <typename T>
T queueArr<T>::dequeue() {
    if (Empty()) {// Check if the queue is empty
        std::cout << "Queue is Empty\n";
        return T(); // Return default value of type T
    }
    T value = q_arr[q_front];// Store the front value
    // Increment the front index in a circular manner
    q_front = (q_front + 1) % q_capacity;
    q_size--;// Decrement the size of the queue
    if (Empty()) Clear();   // Reset the indexes
    return value;// Return the dequeued value
}
#endif // QUEUEARR_H
```

MAIN CPP FILE:

ILOB > mainarr.cpp > main()

```cpp
#include <iostream>
#include "queueArr.h"

int main()
{
    // creating an object
    queueArr<int> q(5);

    q.enqueue(10);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.enqueue(20);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.enqueue(30);

    q.enqueue(40);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.enqueue(50);
```

```
    q.enqueue(50);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.enqueue(60);
    std::cout<<"Tetsing Dequeue"<<std::endl;
    std::cout << "removed: " << q.dequeue() << std::endl;
    std::cout << "removed: " << q.dequeue() << std::endl;

    // testing clear
    q.Clear();
    std::cout << "Testing Clear" << std::endl;
    std::cout << "is the queue empty? "
              << (q.Empty() ? "yes" : "no")
              << std::endl;

    return 0;
}
```

TABLE 3: ILOB (ARRAY-BASED QUEUE)

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

CODE	OUTPUT
Header File: Supple > C queuejobs.h > Job `#ifndef QUEUEJOBS_H` `#define QUEUEJOBS_H` `#include <iostream>` `#include <string>` `// Class Job` `class Job {` `private:` `  int numID;` `  std::string userName;` `  int pageNum;` `public:` `  Job(int newID = 0, std::string newName = "", int numPage = 0) {` `  numID = newID;` `  userName = std::move(newName);` `  pageNum = numPage;` `  }` `  void display() const {` ` std::cout << "[Job " << numID << "	" << userName << "	" << pageNum << " pages]";` ` }` `};`  `// Queue node class` `template <typename T>` `class qNode {` `public:` ` T data;`	upplementary.cpp -o supplementary 1. Add new job 2. Exit Enter choice: 1 Enter Username: James Enter number of pages: 2 1. Add new job 2. Exit Enter choice: 1 Enter Username: Jane Enter number of pages: 3 1. Add new job 2. Exit Enter choice: 1 Enter Username: Sam Enter number of pages: 4 1. Add new job 2. Exit Enter choice: 2

```
class qNode {
    qNode* next;
};

// create new node
template <typename T>
qNode<T>* new_node(T newData) {
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

// Printer class
template <typename T>
class Printer {
public:
    void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr) {
        qNode<T>* newNode = new_node(newData);

        if ((*frontPtr) == nullptr && (*backPtr) == nullptr) {
            *frontPtr = newNode;
            *backPtr = newNode;
            return;
        }

        (*backPtr)->next = newNode;
        *backPtr = newNode;
```

```
    void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr) {
        if ((*frontPtr) == nullptr && (*backPtr) == nullptr) {
            std::cout << "The queue is empty" << std::endl;
            return;
        }

        qNode<T>* deleteTemp = *frontPtr;

        if ((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr) {
            *frontPtr = nullptr;
            *backPtr = nullptr;
            delete deleteTemp;
            return;
        }

        *frontPtr = (*frontPtr)->next;
        deleteTemp->next = nullptr;
        delete deleteTemp;
    }

    void displayQ(qNode<T>** frontPtr, qNode<T>** backPtr) {
        if ((*frontPtr) == nullptr && (*backPtr) == nullptr) {
            std::cout << "The queue is empty" << std::endl;
            return;
```

```
        }

        qNode<T>* curr = *frontPtr;
        while (curr != nullptr) {
            curr->data.display();
            if (curr->next != nullptr) {
                std::cout << " -> ";
            }
            curr = curr->next;
        }
        std::cout << std::endl;
    }
};

#endif
```

```
Queue:
[Job 1 | James | 2 pages] -> [Job 2 | Jane | 3 pages] -> [Job 3 | Sam | 4 pages]

Processing the top...

Queue:
[Job 2 | Jane | 3 pages] -> [Job 3 | Sam | 4 pages]

Processing the top...

Queue:
[Job 3 | Sam | 4 pages]

Processing the top...

Nothing left to print!
PS C:\Users\LENOVO\Downloads\MACASAET_QUEUE\Supple>
```

MAIN CPP FILE:

```cpp
#include <iostream>
#include "queuejobs.h"

int main() {
    Printer<Job> myPrinter;
    qNode<Job>* front = nullptr;
    qNode<Job>* back = nullptr;
    std::string name;
    int pageNum;
    int choice;
    int jobID = 1;
    while (true) {
        std::cout << "1. Add new job\n2. Exit\n";
        std::cout << "Enter choice: ";
        std::cin >> choice;
        if (choice == 1) {
            std::cout << "Enter Username: ";
            std::cin >> name;
            std::cout << "Enter number of pages: ";
            std::cin >> pageNum;
            Job job(jobID++, name, pageNum);
            myPrinter.enqueue(job, &front, &back);
        }
        else if (choice == 2) {
          while(front != nullptr && back != nullptr){
            std::cout << "\nQueue:\n";
            myPrinter.displayQ(&front, &back);
            std::cout << "\nProcessing the top...\n";
            myPrinter.dequeue(&front, &back);
          }
          std::cout << "\nNothing left to print!\n";
          break;
        }
    }
    return 0;
}
```

Defend your choice of internal representation: Why did you use arrays or linked lists?

- I used a linked list approach to queues because the amount you can put in it is not fixed unlike in an array. Because of this, using a linked list is the ideal approach for something like a printer where people can continuously put papers in and print. An array is good when you know how much paper you want to print but a linked list is ideal when you don't know how many papers people will put in the printer over time.

CONCLUSION/ANALYSIS:

- There are 3 classes in the header file, one is class Job which takes in information like Job ID, Username, and number of pages. Second is qNode which defines the characteristics of the node, and class Printer which adds(enqueues) papers, processes(dequeue) papers, and outputs(display) the current queue. There is also the new_node function which creates a new node so that the values of Job will be put in it. When you run the program it asks you if you want to add a job or exit. If you pick 1 then you will have to enter your name and pages then those will be put as values inside the parameter of the Job class alongside the Job ID. Since Printer is inheriting the class Job then your inputs will be enqueued when you call the enqueue function inside the Printer class. Multiple jobs can be enqueued. Then using a while loop the printer displays the current queue then dequeues the top of the queue. Once the queue is empty, the program will print that nothing is left to print.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:
Google Gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Code assistance

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: I used it if when I tried to debug my program so many times and I give then I would ask the help of AI to help me debug.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.